

UNIVERSITÀ DEGLI STUDI DE L'AQUILA

DIPARTIMENTO DISIM

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Analisi di Hash Tables

Analisi e confronto di hash tables con varie tecniche di gestione collisioni e resize strategies

Autore:

Lorenzo D'Angelo

Matricola 302226

Anno Accademico 2025–2026

Abstract

Questo lavoro analizza la realizzazione di hash table in C++, con particolare attenzione a diverse tecniche di gestione delle collisioni e di ridimensionamento (resize). Nei primi capitoli viene presentato il problema affrontato e l'implementazione sviluppata in C++. Successivamente, viene descritto un possibile test di correttezza e il sistema di testing adottato. Infine, viene illustrato il processo per l'elaborazione dei dati e la generazione dei grafici, con una discussione dei risultati ottenuti.

Indice

Abstract	i
1 Introduzione	1
1.1 Obiettivi	1
1.2 Struttura del documento	1
2 Analisi del problema	3
2.1 L'algoritmo	3
2.2 Strategie di gestione conflitti	4
2.2.1 Open Addressing: Linear Probing	4
2.2.2 Open Addressing: Double Hashing	4
2.2.3 Chaining	5
2.2.4 Cuckoo Hashing	6
2.3 Strategie di Resizing	7
2.3.1 Double	7
2.3.2 Constant	7
2.3.3 Dynamic	7
3 Implementazione	9
3.1 Classe template	9
3.2 Hash	10
3.3 Data Structures	12
3.3.1 Open Addressing	12
3.3.2 Chaining	12
3.3.3 Cuckoo Hashing	13
3.4 Operazioni	13
4 Correttezza	15
5 Testing	19
5.1 Metriche	19

5.2	Struttura dei test	21
6	Elaborazione & Analisi	28
6.1	Elaborazione dei dati	28
6.2	Analisi dei dati	31
6.2.1	Hash Data	31
6.2.2	Operation Data	41
6.2.3	Mixed Data	47
7	Conclusioni	53
8	Uso delle LLM	55

Capitolo 1

Introduzione

1.1 Obiettivi

In questo lavoro si procederà ad analizzare la struttura dati Hash Table, studiandone le differenze prestazionali al variare del tipo di dato, delle tecniche di gestione dei conflitti e delle strategie di resizing.

Viene proposta una possibile implementazione in C++, controllata attraverso un test di correttezza presentato più avanti, che verrà utilizzata come base per analizzare quattro tecniche di gestione delle collisioni: `OPEN_ADDRESSING_LINEAR_PROBING`, `OPEN_ADDRESSING_DOUBLE_HASHING`, `CHAINING` e `CUCKOO`.

Verranno inoltre considerate tre strategie di resizing: `DOUBLE`, `CONSTANT` e `DYNAMIC`, che saranno spiegate nei capitoli successivi.

1.2 Struttura del documento

Il documento del progetto è costituito dai seguenti contenuti:

- Analisi e descrizione dell'algoritmo Hash Table.
- Presentazione di una possibile implementazione in C++.
- Test di correttezza per verificare l'implementazione in C++.
- Creazione dell'apparato di testing e realizzazione di grafici per la successiva analisi.
- Elaborazione e analisi dei risultati ottenuti attraverso il testing.
- Conclusioni.

- Use LLM.

Capitolo 2

Analisi del problema

2.1 L'algoritmo

Le hash table nascono per rispondere all'esigenza di memorizzare grandi quantità di dati garantendo, nel caso medio, una complessità temporale pari a $O(1)$ per le operazioni di **search**, **insert** e **delete**.

L'idea di base consiste nel memorizzare dati organizzati in coppie chiave-valore all'interno di un array. Tale struttura consente l'accesso diretto agli elementi in tempo costante, sfruttando un indice calcolato tramite una funzione apposita, detta funzione di hash.

La funzione di hash ha il compito di trasformare una chiave in un indice dell'array in tempo $O(1)$. Una volta ottenuto l'indice, è possibile accedere direttamente alla posizione corrispondente nell'array, mantenendo quindi il tempo di accesso costante.

Affinché tutte le operazioni possano essere eseguite in tempo $O(1)$ anche nel caso peggiore, sarebbe necessario disporre di una funzione di hash perfetta, in grado di evitare completamente le collisioni (ovvero i casi in cui chiavi diverse producono lo stesso indice), e di uno spazio di memoria illimitato, così da non dover mai effettuare operazioni di ridimensionamento della struttura.

Tuttavia, tali condizioni ideali non sono realisticamente raggiungibili. Nella pratica, è necessario adottare opportune tecniche di gestione delle collisioni e strategie di ridimensionamento (resize). Grazie a queste soluzioni, è possibile garantire che, nel caso medio, le operazioni fondamentali mantengano una complessità $O(1)$, mentre nel caso peggiore possano degradare fino a $O(n)$.

2.2 Strategie di gestione conflitti

Nel lavoro effettuato sono state prese in considerazione quattro strategie di gestione dei conflitti, ovvero OPEN_ADDRESSING_LINEAR_PROBING, OPEN_ADDRESSING_DOUBLE_HASHING, CHAINING e CUCKOO.

2.2.1 Open Addressing: Linear Probing

L'open addressing è una tecnica di risoluzione delle collisioni in cui ogni elemento viene memorizzato direttamente nell'array dei bucket e la gestione delle collisioni avviene tramite probing (sondaggio).

Il *linear probing* consiste nell'esaminare i bucket in modo sequenziale, partendo dalla posizione calcolata dalla funzione di hash e proseguendo linearmente (posizioni successive), finché non si trova la posizione desiderata.

Quando si deve inserire un nuovo elemento, i bucket vengono esaminati a partire dalla posizione calcolata dalla funzione di hash e si procede linearmente fino a trovare uno slot libero.

Quando si cerca un elemento, si segue la stessa sequenza di probing, finché non si trova il record desiderato oppure uno slot vuoto, il che indica che l'elemento non è presente nella tabella.

Questa tecnica ha complessità nel caso medio pari a $O(1)$, mentre nel caso peggiore è $O(n)$, in quanto potrebbe essere necessario scandire l'intero array. Le prestazioni dipendono fortemente dal fattore di carico $\alpha = \frac{n}{m}$ della tabella, che rappresenta il rapporto tra il numero di celle occupate e il numero di posizioni disponibili. Tipicamente, tale valore viene mantenuto nell'intervallo 0.6–0.75 per garantire buone prestazioni.

2.2.2 Open Addressing: Double Hashing

L'open addressing è una tecnica di risoluzione delle collisioni in cui ogni elemento viene memorizzato direttamente nell'array dei bucket e la gestione delle collisioni avviene tramite probing (sondaggio).

Il *double hashing* è una tecnica di probing avanzata, in cui la sequenza di posizioni da esaminare dipende da due funzioni di hash. La prima funzione di hash $h_1(k)$ determina la posizione iniziale del bucket, mentre la seconda funzione $h_2(k)$ definisce l'incremento per gli step successivi. Le posizioni da esaminare vengono calcolate come:

$$(h_1(k) + i \cdot h_2(k)) \mod m, \quad i = 0, 1, 2, \dots$$

Quando si deve inserire un nuovo elemento, si parte dalla posizione calcolata da $h_1(k)$ e, in caso di collisione, si procede seguendo la sequenza definita dalla combinazione di h_1 e h_2 fino a trovare uno slot libero.

Quando si cerca un elemento, si segue la stessa sequenza di probing. La ricerca termina quando si trova il record desiderato oppure uno slot vuoto, indicando che l'elemento non è presente nella tabella.

È fondamentale che $h_2(k)$ sia coprima con la dimensione dell'array m , altrimenti non tutte le posizioni disponibili potrebbero essere visitabili. Se $h_2(k)$ e m non sono coprime, alcune posizioni rimarrebbero escluse dalla sequenza di probing, riducendo l'efficienza della tabella e aumentando il rischio di collisioni non risolvibili.

Il *double hashing* riduce significativamente il problema del *primary clustering* presente nel linear probing, poiché la sequenza di probing varia a seconda della chiave.

Questa tecnica ha complessità nel caso medio pari a $O(1)$, mentre nel caso peggiore può arrivare a $O(n)$. Le prestazioni dipendono fortemente dal fattore di carico $\alpha = \frac{n}{m}$, che rappresenta il rapporto tra il numero di celle occupate e il numero di posizioni disponibili. Tipicamente, tale valore viene mantenuto nell'intervallo 0.6–0.75 per garantire buone prestazioni.

2.2.3 Chaining

Il *chaining* è una tecnica di risoluzione delle collisioni in cui ogni bucket dell'array non contiene un solo elemento, ma una lista (tipicamente una linked list) di tutti gli elementi che hashano nella stessa posizione.

Quando si deve inserire un nuovo elemento, si calcola la posizione tramite la funzione di hash $h(k)$ e si aggiunge l'elemento alla lista presente in quel bucket. In questo modo, anche se più elementi hashano nella stessa posizione, tutti possono essere memorizzati senza dover spostare altri elementi nell'array.

Quando si cerca un elemento, si calcola la posizione tramite $h(k)$ e si scorre la lista associata al bucket fino a trovare il record desiderato o fino a raggiungere la fine della lista, che indica che l'elemento non è presente.

Il chaining evita il problema del *primary clustering* presente nelle tecniche di open addressing, poiché ogni bucket può contenere un numero illimitato di elementi hashati nella stessa posizione. Tuttavia, le prestazioni dipendono dalla lunghezza media delle liste.

La complessità nel caso medio per ricerca, inserimento o cancellazione è $O(1 + \alpha)$, dove $\alpha = n/m$ è il fattore di carico della tabella, cioè il rapporto tra il numero totale di elementi n e il numero di bucket m . Nel caso peggiore, se tutti gli elementi hashano nello stesso bucket, la complessità può arrivare

a $O(n)$, ma questo scenario è raro se la funzione di hash distribuisce bene gli elementi.

Questa tecnica è particolarmente efficace quando il numero di elementi da memorizzare non è noto a priori o quando il fattore di carico può superare 1, cosa che non è possibile con l'open addressing senza ridimensionare la tabella. Tipicamente, tale valore viene mantenuto nell'intervallo 1–3 per garantire buone prestazioni.

2.2.4 Cuckoo Hashing

Il *Cuckoo Hashing* è una tecnica di risoluzione delle collisioni che utilizza due (o più) funzioni di hash e due tabelle separate. Ogni elemento può essere memorizzato in una sola posizione per ciascuna funzione di hash, e in caso di collisione un elemento “scaccia” quello già presente nella posizione, il quale viene reinserito nella sua posizione alternativa calcolata dalla seconda funzione di hash.

Quando si deve inserire un nuovo elemento, si calcola la prima posizione tramite $h_1(k)$. - Se lo slot è libero, l'elemento viene inserito. - Se lo slot è occupato, l'elemento corrente viene “espulso” e inserito nella sua posizione alternativa calcolata da $h_2(k)$. Questo processo di spostamento può ripetersi fino a che tutti gli elementi trovano una posizione libera o fino a raggiungere un limite massimo di spostamenti. In quest'ultimo caso, la tabella viene ridimensionata o le funzioni di hash vengono ricalcolate.

Quando si cerca un elemento, basta controllare le posizioni calcolate da $h_1(k)$ e $h_2(k)$; se l'elemento non è presente in nessuna delle due, significa che non è nella tabella. Questa caratteristica garantisce che la ricerca sia sempre $O(1)$ nel caso peggiore.

Il Cuckoo Hashing evita il problema del *primary clustering* presente nel linear probing, e mantiene un fattore di carico relativamente alto senza compromettere le prestazioni di ricerca. Tuttavia, le operazioni di inserimento possono richiedere più spostamenti e, in casi rari, il ridimensionamento della tabella.

La complessità media di ricerca è $O(1)$, così come nel caso peggiore, mentre l'inserimento ha complessità media $O(1)$ ma nel caso peggiore può richiedere più operazioni di spostamento o un ridimensionamento della tabella. Tipicamente, il fattore di carico $\alpha = n/m$ viene mantenuto nell'intervallo 0.4–0.6 per garantire buone prestazioni e ridurre il numero di cicli di spostamento.

2.3 Strategie di Resizing

Nel lavoro effettuato sono state prese in considerazione tre strategie di *resize*, ovvero: **DOUBLE**, **CONSTANT** e **DYNAMIC**. Queste strategie definiscono come la dimensione della tabella hash viene aumentata quando il fattore di carico supera un certo valore soglia.

2.3.1 Double

La strategia **DOUBLE** consiste nel raddoppiare la dimensione della tabella ogni volta che il fattore di carico supera una soglia prefissata. Quando avviene il *resize*, tutti gli elementi vengono re-inseriti nella nuova tabella utilizzando la stessa funzione di hash. Questo approccio è semplice e garantisce che il fattore di carico torni ad essere basso dopo ogni ridimensionamento. Tuttavia, il costo dell'operazione di raddoppio può essere elevato, poiché tutti gli elementi devono essere reinseriti.

2.3.2 Constant

La strategia **CONSTANT** prevede di aumentare la dimensione della tabella di un valore fisso ogni volta che il fattore di carico supera la soglia. Questo approccio distribuisce il costo del *resize* in modo più uniforme rispetto al raddoppio, ma può comportare più operazioni di *resize* se la crescita del numero di elementi è elevata. È utile quando si prevede una crescita lineare del numero di elementi nella tabella.

2.3.3 Dynamic

La strategia **DYNAMIC** prevede il raddoppio della dimensione della tabella, come nella strategia **DOUBLE**, ma lo spostamento degli elementi avviene in modo incrementale. In pratica, non tutti gli elementi vengono copiati nella nuova tabella immediatamente, ma viene spostato un *chunk* di elementi ad ogni operazione di inserimento o ricerca, fino a completare il trasferimento di tutti gli elementi dalla vecchia tabella a quella nuova.

Quando si cerca un elemento durante il periodo di trasferimento, si procede nel seguente modo:

- Si cerca prima nella nuova tabella.
- Se non è presente, si cerca nella vecchia tabella.
- Se l'elemento è trovato nella vecchia tabella, viene copiato nella nuova.

Questa tecnica permette di distribuire il costo del ridimensionamento su più operazioni, evitando picchi di tempo elevati durante un singolo inserimento. Mantiene le prestazioni di ricerca e inserimento prossime a $O(1)$ anche durante il processo di resizing, rendendola più efficiente rispetto al raddoppio immediato della strategia **DOUBLE**.

Capitolo 3

Implementazione

3.1 Classe template

L'implementazione dell'algoritmo è stata effettuata in C++, attraverso l'utilizzo di una classe `template`, in modo da garantire l'utilizzo della classe con diversi tipi di chiavi e dati. La classe è stata strutturata con degli enum in modo da andare a configurare opportunamente l'istanza con le strategie scelte per la hash table.

Listing 3.1: HashTable class

```
1  enum class OVERFLOW {
2      OPEN_ADDRESSING_LINEAR_PROBING ,
3      OPEN_ADDRESSING_DOUBLE_HASHING ,
4      CHAINING ,
5      CUCKOO
6  };
7  enum class HASH {
8      MODE_1 ,
9      MODE_2
10 };
11 enum class RESIZE {
12     DOUBLE ,
13     CONSTANT ,
14     DYNAMIC
15 };
16
17 template <typename Tkey, typename Tvalue>
18 class HashTable {
19     public:
20         HashTable(
```

```

21         OVERFLOW overflow = OVERFLOW::
22             OPEN_ADDRESSING_LINEAR_PROBING,
23         HASH hash = HASH::MODE_1,
24         RESIZE resize = RESIZE::DOUBLE,
25         std::size_t table_length = 256,
26         double max_load_factor = 0.75
27     };
}

```

3.2 Hash

Per garantire un corretto funzionamento ed un corretto hashing al variare del tipo delle chiavi da inserire, queste vengono prima convertite attraverso l'apposita funzione `KeyConvert` e successivamente hashate e gestite secondo la strategia di collisione selezionata:

Listing 3.2: `KeyConvert` function

```

1  template<typename K>
2  struct KeyConvert {
3      static std::size_t hash(const K& key) {
4          if constexpr (std::is_integral_v<K> || std::
5              is_same_v<K, char>) {
6              return static_cast<std::size_t>(key);
7          } else if constexpr (std::is_same_v<K, std::
8              string>) {
9              // stringhe
10             std::size_t h = 0;
11             for (char c : key) h = h * 31 + static_cast<
12                 unsigned char>(c);
13             return h;
14         } else {
15             static_assert(sizeof(K) == 0, "Tipo non supportato");
16         }
17     }
18 };

```

Il progetto utilizza tre diverse hash function, ciascuna con uno scopo specifico:

1. **Prima hash function (h1):** Genera l'indice principale della tabella tramite l'operazione di modulo tra il valore hash della chiave e la lunghezza della tabella.

Listing 3.3: First hash function

```
1  template <typename Tkey, typename Tvalue>
2  std::size_t HashTable<Tkey, Tvalue>::h1(const
    Tkey& key) const {
3      std::size_t h = KeyConvert<Tkey>::hash(key);
4      // modulo table_length
5      return h % table_length;
6  }
```

2. **Seconda hash function (h2):** Può essere selezionata tramite apposito parametro durante la creazione della tabella. Viene utilizzata sia per l'hashing di base alternativo sia come seconda hash function nella strategia Cuckoo. Nel caso in cui sia selezionata per la seconda tabella della strategia Cuckoo, h1 viene usata come fallback. La funzione moltiplica il valore hash della chiave per una costante molto grande a e poi applica l'operazione di modulo con la lunghezza della tabella. Questo approccio genera una distribuzione più uniforme degli indici rispetto a h1.

Listing 3.4: Second hash function

```
1  template <typename Tkey, typename Tvalue>
2  std::size_t HashTable<Tkey, Tvalue>::h2(const
    Tkey& key) const {
3      std::size_t h = KeyConvert<Tkey>::hash(key);
4      constexpr std::size_t a =
        11400714819323198485ull;
5      return (h * a) % table_length;
6  }
```

3. **Terza hash function (h3):** Specifica per la strategia di double hashing, calcola l'incremento di probing in caso di collisioni. La funzione utilizza un numero primo R immediatamente inferiore alla dimensione della tabella per garantire che l'incremento e la lunghezza della tabella siano coprimi, così da poter esplorare tutte le posizioni dell'array.

Listing 3.5: Hash function for double hashing

```
1  template <typename Tkey, typename Tvalue>
```

```

2      std::size_t HashTable<Tkey, Tvalue>::h3(const
      Tkey& key) const {
3          std::size_t h = KeyConvert<Tkey>::hash(key);
4          std::size_t R = previousPrime(table_length -
              1);
5          return R - (h % R);
6      }

```

In questo modo, la tabella può supportare diversi tipi di chiave e diverse strategie di collisione, mantenendo le operazioni di inserimento, ricerca e cancellazione efficienti.

3.3 Data Structures

3.3.1 Open Addressing

Per l'open addressing, la struttura dati utilizzata per l'implementazione della hash table è un `std::vector<Slot>`, ovvero un vettore di oggetti `Slot`. Ogni `Slot` è caratterizzato da due campi per la chiave e il valore, più un campo `state` che indica se la cella è vuota, occupata o è stata rimossa. Questo campo aggiuntivo è fondamentale per preservare la logica di ricerca dell'open addressing anche dopo la cancellazione di elementi.

Listing 3.6: Open Addressing Data Structure

```

1  enum class SlotState { EMPTY, OCCUPIED, DELETED };
2
3  struct Slot {
4      SlotState state = SlotState::EMPTY;
5      Tkey key;
6      Tvalue value;
7  };
8
9  std::vector<Slot> table_open;

```

3.3.2 Chaining

Per il chaining, la struttura dati utilizzata è un `std::vector<std::vector<std::pair<Tkey, Tvalue>>>`, ovvero un vettore di vettori che rappresentano i bucket della tabella hash. Ogni bucket contiene una lista di coppie chiave-valore corrispon-

denti agli elementi presenti, permettendo di gestire più elementi nella stessa posizione senza perdere dati.

Listing 3.7: Chaining Data Structure

```
1 std::vector<std::vector<std::pair<Tkey, Tvalue>>>  
   table_chaining;
```

3.3.3 Cuckoo Hashing

Per il Cuckoo Hashing, la struttura dati è costituita da due `std::vector<Slot>`, duplicando la struttura utilizzata per l'open addressing. Ogni `Slot` contiene la chiave, il valore e un campo che indica se la cella è vuota. Le due tabelle separate consentono il funzionamento della strategia Cuckoo, dove ogni elemento può essere spostato da una tabella all'altra in caso di collisione.

Listing 3.8: Cuckoo Data Structure

```
1 std::vector<Slot> table_cuckoo_1;  
2 std::vector<Slot> table_cuckoo_2;
```

Durante l'inserimento, se la posizione calcolata tramite la prima funzione di hash è occupata, l'elemento presente viene espulso e reinserito nella posizione corrispondente della seconda tabella, garantendo che ogni elemento abbia sempre due possibili posizioni.

3.4 Operazioni

Le seguenti funzioni rappresentano le principali operazioni disponibili nella classe `HashTable`:

Listing 3.9: Main Operations

```
1 public:  
2     bool insert(const Tkey& key, const Tvalue& value);  
3     bool update(const Tkey& key, const Tvalue& value);  
4     bool remove(const Tkey& key);  
5     bool exist(const Tkey& key);  
6     std::optional<Tvalue> get(const Tkey& key);  
7  
8 private:  
9     bool resizeTable();  
10    void dynamicCopy();
```

```

11     std::optional<std::size_t> find(const Tkey& key);
12     std::optional<std::size_t> _insert(const Tkey& key,
        const Tvalue& value);

```

Le operazioni principali offerte dalla struttura dati sono:

- **insert**: inserisce una nuova coppia chiave-valore nella tabella. Se la chiave è già presente, l'operazione si comporta come una **update**.
- **update**: aggiorna il valore associato a una chiave esistente.
- **remove**: elimina un elemento dalla tabella, gestendo correttamente la struttura dati a seconda della strategia di collisione utilizzata.
- **exist**: verifica la presenza di una chiave all'interno della tabella.
- **get**: restituisce il valore associato a una chiave, se presente, utilizzando `std::optional` per gestire il caso in cui la chiave non esista.

Tutte le operazioni di accesso si appoggiano alla funzione interna **find**, che implementa la logica di ricerca in base alla strategia di risoluzione delle collisioni selezionata (open addressing, chaining o cuckoo hashing). Questa funzione restituisce la posizione dell'elemento, se presente, oppure un valore nullo in caso contrario.

La funzione privata **_insert** gestisce nel dettaglio la logica di inserimento, includendo la gestione delle collisioni e il posizionamento corretto dell'elemento nella struttura dati. Questa funzione viene utilizzata internamente da **insert** e può essere adattata a seconda della strategia adottata.

Il fattore di carico della tabella viene monitorato durante le operazioni di inserimento: se il valore supera la soglia **max_load_factor**, viene invocata la funzione **resizeTable**, che provvede ad aumentare la dimensione della tabella e a riorganizzare gli elementi.

Nel caso della strategia **DYNAMIC**, il ridimensionamento non avviene in modo immediato: la funzione **dynamicCopy** viene utilizzata per spostare progressivamente gli elementi dalla vecchia tabella a quella nuova, distribuendo il costo del **resize** su più operazioni e migliorando le prestazioni complessive.

Capitolo 4

Correttezza

Il test di correttezza è stato implementato in C++ all'interno del file `correctness.cpp`.

Sono state considerate quattro diverse tipologie di dati: `integer`, `random strings`, `real words` e `hybrid`, dove la chiave è di tipo `real word` e il valore è di tipo `integer`.

Nel file è stata implementata una sezione di test per ognuna delle quattro tipologie di dato, con l'obiettivo di verificare la correttezza delle operazioni fondamentali e di collezionare eventuali errori riscontrati durante l'esecuzione.

Listing 4.1: Integer test

```
1 // Integer
2 {
3     int errors = 0;
4     for(int x=0; x<20; x++) {
5         for(int i=0; i<6; i++) {
6             int current_error = intTest(std::pow(10, i));
7             std::cout << "Partial Operation Failed: " <<
8                 current_error << std::endl;
9             std::cout << "-----"
10                 <<std::endl;
11             errors += current_error;
12         }
13     }
14     arr_error[0] += errors;
15     std::cout << "-----" <<std::endl;
16     std::cout << "-----" <<std::endl;
17     endl;
```

```

15     std::cout << "INTEGER_ERRORS:" << errors << std::
        endl;
16     std::cout << "-----" <<std::
        endl;
17     std::cout << "-----" <<std::
        endl;
18 }

```

Ogni tipologia di dato utilizza una funzione di test dedicata, che si occupa di verificare il comportamento della struttura dati su diverse configurazioni di hash table. Per ciascuna configurazione viene creata una nuova istanza della classe `HashTable`, parametrizzata con le specifiche modalità di gestione delle collisioni, funzione di hash e strategia di ridimensionamento.

Il test viene inoltre ripetuto più volte, variando il numero di elementi utilizzati, al fine di ridurre l'impatto della casualità e ottenere risultati più affidabili.

Listing 4.2: Integer test function

```

1  /*
2  * HashTable:
3  *   OVERFLOW::OPEN_ADDRESSING_LINEAR_PROBING
4  *   HASH::MODE_1
5  *   RESIZE::DOUBLE
6  *
7  * Key::INTEGER
8  * Value::INTEGER
9  */
10 {
11     std::cout << "-----" <<std::
        endl;
12     std::cout << "HashTable:" << std::endl;
13     std::cout << "  OVERFLOW::
        OPEN_ADDRESSING_LINEAR_PROBING" << std::endl;
14     std::cout << "  HASH::MODE_1" << std::endl;
15     std::cout << "  RESIZE::DOUBLE" << std::endl;
16     std::cout << "  Key::INTEGER" << std::endl;
17     std::cout << "  Value::INTEGER" << std::endl;
18     std::cout << "-----" <<std::
        endl;
19
20     HashTable<int, int> HashTable(OVERFLOW::
        OPEN_ADDRESSING_LINEAR_PROBING, HASH::MODE_1,
        RESIZE::DOUBLE);

```

```

21     errors += intTestRoutine(HashTable, elements);
22 }

```

Per ogni configurazione viene eseguita una routine specifica, differenziata in base alla tipologia di dato, che verifica la correttezza delle operazioni fondamentali della hash table.

In particolare:

- **Insert:** viene verificato che ogni elemento inserito sia effettivamente memorizzato nella struttura e accessibile tramite operazione di ricerca (`find`).
- **Update:** viene controllato che i valori associati alle chiavi siano correttamente aggiornati.
- **Remove:** viene verificato che gli elementi rimossi non siano più accessibili e che l'operazione non comprometta l'accesso agli altri dati presenti nella struttura.

Listing 4.3: Integer test routine for insert operation

```

1  const int ELEMENTS = elements;
2  int errors = 0;
3  // Generatore di numeri casuali
4  std::vector<std::pair<int, int>> numbers;
5  numbers.reserve(ELEMENTS);
6
7  std::random_device rd;
8  std::mt19937 gen(rd());
9  std::uniform_int_distribution<> dist(1, ELEMENTS*10);
10
11 int value = 0;
12 for (int i = 0; i < ELEMENTS; ++i) {
13     value = dist(gen);
14     numbers.emplace_back(value, value);
15 }
16
17 // Inserts
18 {
19     std::cout << "-----" <<std:::
20         endl;
21     std::cout << "INSERT" <<std:::endl;
22     std::cout << "-----" <<std:::
23         endl;

```

```

22     std::vector<std::pair<int, int>> failed;
23     failed = insert(HashTable, numbers);
24     errors += failed.size();
25     // Exist
26     failed = exist(HashTable, numbers);
27     errors += failed.size();
28     // Get
29     failed = get(HashTable, numbers);
30     errors += failed.size();
31 }

```

Durante l'esecuzione dei test, il numero di errori riscontrati viene accumulato e riportato per ciascuna tipologia di dato e configurazione testata.

L'esecuzione complessiva dei test non ha evidenziato errori, confermando la correttezza dell'implementazione delle operazioni fondamentali della hash table nelle diverse configurazioni considerate.

Listing 4.4: Correctness test result

```

1  -----
2  -----
3  TOTAL  ERRORS: 0
4  -----
5  INTEGER ERRORS: 0
6  STRING  ERRORS: 0
7  REAL WORD ERRORS: 0
8  HYBRID  ERRORS: 0
9  -----
10 -----

```

Capitolo 5

Testing

Per la fase di testing sono state sviluppate quattro diverse implementazioni, una per ciascun tipo di dato precedentemente descritto, ciascuna organizzata all'interno della rispettiva cartella.

Sono stati condotti due test distinti: il primo analizza l'andamento delle prestazioni al crescere della dimensione dell'input, considerando valori pari alle potenze di 10; il secondo, invece, studia l'evoluzione degli stessi parametri al raddoppio della dimensione dell'input.

Nel seguito viene presentata la struttura e le scelte adottate per il test basato sulle potenze di 10; tuttavia, le considerazioni risultano valide anche per il test con incremento per raddoppio.

5.1 Metriche

Durante l'esecuzione dei test vengono generati due file distinti:

Listing 5.1: File di output dei test

```
1 std::pair<std::ofstream, std::ofstream> setup() {
2     std::ofstream op_file("op_data.csv");
3     std::ofstream hash_file("hash_data.csv");
4
5     // Intestazione
6     op_file << "OVERFLOW,HASH,RESIZE,Elements,
7               Max_load_factor,Operazione,Latenza,Load_factor,
8               Memoria,Key_distribution\n";
9     hash_file << "OVERFLOW,HASH,RESIZE,Elements,
10               Max_load_factor,Latenza,Memoria\n";
11
12     return {std::move(op_file), std::move(hash_file)};
```

Il file `hash_data.csv` contiene informazioni aggregate relative a ciascuna istanza di test, memorizzando le caratteristiche della configurazione utilizzata (strategia di gestione delle collisioni, funzione di hash, politica di resize, numero di elementi e fattore di carico massimo) insieme a metriche prestazionali globali quali latenza complessiva e memoria utilizzata.

Il file `op_data.csv`, invece, raccoglie informazioni a grana fine per ogni singola operazione eseguita su ciascuna istanza. Per ogni operazione vengono registrati il tipo di istanza, la tipologia di operazione eseguita, la latenza, la memoria occupata, il fattore di carico corrente e un indice di distribuzione delle chiavi.

La latenza viene misurata come tempo CPU utilizzando la funzione `clock_gettime` con clock `CLOCK_PROCESS_CPUTIME_ID`, che consente di ottenere una misura stabile del tempo di esecuzione del processo, riducendo l'influenza di fattori esterni.

Distribuzione delle chiavi

La distribuzione delle chiavi viene stimata mediante metriche differenti a seconda della strategia di gestione delle collisioni adottata:

- **Open Addressing (Linear Probing e Double Hashing):** viene calcolata la dimensione media dei cluster. Un cluster è definito come una sequenza contigua di celle occupate. L'algoritmo identifica tali sequenze, ne calcola la lunghezza e restituisce la media tra tutte le lunghezze osservate. Valori elevati indicano la presenza di fenomeni di clustering, generalmente associati a un peggioramento delle prestazioni.
- **Chaining:** viene calcolata la deviazione standard della lunghezza dei bucket. Questa misura consente di valutare quanto uniformemente gli elementi siano distribuiti tra le liste: valori bassi indicano una distribuzione omogenea, mentre valori elevati evidenziano squilibri tra i bucket.
- **Cuckoo Hashing:** viene adottato un approccio analogo all'open addressing, calcolando la dimensione media dei cluster su entrambe le tabelle hash utilizzate dalla struttura. Le due tabelle vengono considerate congiuntamente nel processo di individuazione dei cluster.

Si osserva che le metriche adottate non sono direttamente confrontabili tra le diverse strategie, ma risultano utili per analizzare il comportamento interno di ciascun metodo.

5.2 Struttura dei test

Il test esegue una routine variando, per ogni iterazione, il valore del `load_factor`. In particolare, vengono considerate tre configurazioni:

- valore standard (consigliato per la struttura),
- valore inferiore,
- valore superiore.

Listing 5.2: Test con differenti load factor

```
1  std::pair<std::ofstream, std::ofstream> files = setup();
2
3  for(int x=0; x<5; x++) {
4      // Standard load factor
5      for(int i=1; i<5; i++) {
6          std::cout << "-----" <<
7              std::endl;
8          std::cout << "ELEMENT_COUNT:" << std::pow(10, i)
9              <<std::endl;
10         std::cout << "-----" <<
11             std::endl;
12         intTest(std::pow(10, i), files);
13     }
14
15     // Low load factor
16     for(int i=1; i<5; i++) {
17         std::cout << "-----" <<
18             std::endl;
19         std::cout << "ELEMENT_COUNT:" << std::pow(10, i)
20             <<std::endl;
21         std::cout << "-----" <<
22             std::endl;
23         intTest(std::pow(10, i), files, 1);
24     }
25
26     // High load factor
27     for(int i=1; i<5; i++) {
28         std::cout << "-----" <<
29             std::endl;
30         std::cout << "ELEMENT_COUNT:" << std::pow(10, i)
31             <<std::endl;
```

```

24         std::cout << "-----" <<
           std::endl;
25         intTest(std::pow(10, i), files, 2);
26     }
27 }

```

Ogni iterazione invoca una funzione di test che genera una nuova istanza della classe `HashTable` con differenti parametri di configurazione.

Listing 5.3: Configurazione delle istanze

```

1  /*
2  * HashTable:
3  *   OVERFLOW::OPEN_ADDRESSING_LINEAR_PROBING
4  *   HASH::MODE_1
5  *   RESIZE::DOUBLE
6  *
7  * Key::INTEGER
8  * Value::INTEGER
9  */
10 {
11     std::cout << "-----" <<std::
        endl;
12     std::cout << "HashTable:" << std::endl;
13     std::cout << "  OVERFLOW::
        OPEN_ADDRESSING_LINEAR_PROBING" << std::endl;
14     std::cout << "  HASH::MODE_1" << std::endl;
15     std::cout << "  RESIZE::DOUBLE" << std::endl;
16     std::cout << "  Key::INTEGER" << std::endl;
17     std::cout << "  Value::INTEGER" << std::endl;
18     std::cout << "-----" <<std::
        endl;
19
20     HashTable<int, int> HashTable(OVERFLOW::
        OPEN_ADDRESSING_LINEAR_PROBING, HASH::MODE_1,
        RESIZE::DOUBLE, 256, linear_load_factor);
21     intTestRoutine(HashTable, elements, files.first,
        files.second);
22 }
23 {
24     std::cout << "-----" <<std::
        endl;
25     std::cout << "HashTable:" << std::endl;
26     std::cout << "  OVERFLOW::
        OPEN_ADDRESSING_LINEAR_PROBING" << std::endl;

```

```

27     std::cout << "░░HASH::MODE_1" << std::endl;
28     std::cout << "░░RESIZE::DOUBLE" << std::endl;
29     std::cout << "░░Key::INTEGER" << std::endl;
30     std::cout << "░░Value::INTEGER" << std::endl;
31     std::cout << "-----" <<std:::
        endl;
32
33     HashTable<int, int> HashTable(OVERFLOW::
        OPEN_ADDRESSING_LINEAR_PROBING, HASH::MODE_1,
        RESIZE::DOUBLE, 256, linear_load_factor);
34     intTestRoutine(HashTable, elements, files.first,
        files.second, false);
35 }

```

La routine di test procede inizialmente alla generazione dei dati da inserire e successivamente esegue una sequenza di operazioni selezionate in modo pseudo-casuale secondo la seguente distribuzione:

- 40% Exist
- 30% Insert
- 10% Update
- 10% Get
- 10% Remove

Durante l'esecuzione, la routine può operare in due modalità:

- **Modalità analitica:** ogni operazione viene registrata nel file `op_data.csv`;
- **Modalità prestazionale:** le operazioni vengono eseguite senza logging dettagliato, al fine di evitare overhead e misurare accuratamente le prestazioni complessive, salvate in `hash_data.csv`.

Listing 5.4: Test routine

```

1  template <typename K, typename V>
2  void intTestRoutine(HashTable<K, V>& HashTable, int
    elements, std::ofstream& file_hash, std::ofstream&
    file_op, bool mode = true) {
3      const int ELEMENTS = elements;
4      int errors = 0;
5      // Generatore di numeri casuali

```

```

6      std::vector<std::pair<int, int>> numbers;
7      numbers.reserve(ELEMENTS);
8
9      std::random_device rd;
10     std::mt19937 gen(rd());
11     std::uniform_int_distribution<> dist(1, ELEMENTS*10);
12
13     int value = 0;
14     for (int i = 0; i < ELEMENTS; ++i) {
15         value = dist(gen);
16         numbers.emplace_back(value, value);
17     }
18
19     struct timespec start, end;
20     if(!mode) {
21         clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &start);
22     }
23
24     std::size_t counter = 0;
25     while(counter < numbers.size()) {
26         std::uniform_int_distribution<> dist2(0, 9);
27         switch(dist2(gen)) {
28             case 0:
29             case 1:
30             case 2:
31             case 3: {
32                 // Exist
33                 if(counter > 0) {
34                     std::uniform_int_distribution<>
35                     dist_temp(0, numbers.size() - 1);
36                     exist(HashTable, numbers[dist_temp(
37                         gen)].first, file_op, elements,
38                         mode);
39                 }
40                 break;
41             }
42             case 4:
43             case 5:
44             case 6: {
45                 // Insert
46                 insert(HashTable, numbers[counter],
47                     file_op, elements, mode);
48                 counter++;

```

```

45         break;
46     }
47     case 7: {
48         // Update
49         if(counter > 0) {
50             std::uniform_int_distribution<>
51                 dist_temp(0, numbers.size() - 1);
52             update(HashTable, std::make_pair(
53                 numbers[dist_temp(gen)].first,
54                 dist_temp(gen)), file_op, elements,
55                 mode);
56         }
57         break;
58     }
59     case 8: {
60         // Get
61         if(counter > 0) {
62             std::uniform_int_distribution<>
63                 dist_temp(0, numbers.size() - 1);
64             get(HashTable, numbers[dist_temp(gen)
65                 ].first, file_op, elements, mode);
66         }
67         break;
68     }
69     case 9: {
70         // Remove
71         if(counter > 0) {
72             std::uniform_int_distribution<>
73                 dist_temp(0, numbers.size() - 1);
74             remove(HashTable, numbers[dist_temp(
75                 gen)].first, file_op, elements,
76                 mode);
77         }
78         break;
79     }
80 }
81
82 if(!mode) {
83     clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &end);
84     double cpu_time = (end.tv_sec - start.tv_sec) + (
85         end.tv_nsec - start.tv_nsec) / 1e9;
86 }

```

```

78         csvHashPrint(
79             file_hash,
80             HashTable.getOverflow(),
81             HashTable.getHash(),
82             HashTable.getResize(),
83             elements,
84             HashTable.getMaxLoadFactor(),
85             cpu_time,
86             HashTable.maxMemoryUsage()
87         );
88     }
89 }

```

Ogni operazione eseguita sulla struttura (inserimento, ricerca, aggiornamento e rimozione) misura il tempo di esecuzione e, se abilitato, registra i risultati nel file `op_data.csv`, includendo anche metriche dinamiche come il fattore di carico e la distribuzione delle chiavi.

Listing 5.5: Esempio operazione

```

1  template <typename K, typename V>
2  void insert(HashTable<K, V>& hashTable, const std::pair<K
   , V>& arr, std::ofstream& file, int elements, bool
   print = true) {
3      if(print) {
4          struct timespec start, end;
5          clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &start);
6
7          hashTable.insert(arr.first, arr.second);
8
9          clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &end);
10         double cpu_time = (end.tv_sec - start.tv_sec) + (
            end.tv_nsec - start.tv_nsec) / 1e9;
11
12         csvOperationPrint(
13             file,
14             hashTable.getOverflow(),
15             hashTable.getHash(),
16             hashTable.getResize(),
17             elements,
18             hashTable.getMaxLoadFactor(),
19             "INSERT",
20             cpu_time,
21             hashTable.getLoadFactor(),
22             hashTable.memoryUsage(),

```

```
23         hashTable.getKeyDistribution()  
24     );  
25     } else {  
26         hashTable.insert(arr.first, arr.second);  
27         hashTable.memoryUsage();  
28     }  
29 }
```

Il build dell'apparato di testing viene seguito con i seguenti parametri di ottimizzazione

Listing 5.6: Parametri compilazione

```
1 g++ -O3 -march=native -flto -std=c++20 intTest.cpp -o ./  
   build/intTest
```

Capitolo 6

Elaborazione & Analisi

6.1 Elaborazione dei dati

I dati ottenuti durante la fase di testing, salvati nei file `hash_data.csv` e `op_data.csv` per il test con potenze di 10 e i file `hash_data_double.csv` e `op_data_double.csv` per il test al raddoppio, sono stati successivamente rielaborati mediante l'utilizzo del linguaggio Python. In particolare, sono state impiegate le librerie `pandas` per la manipolazione e l'analisi dei dati e `matplotlib.pyplot` per la generazione dei grafici.

Sono stati sviluppati quattro script principali:

- `grapher_hash.py` e `grapher_op.py`, dedicati all'analisi dei dati relativi a ciascuna singola tipologia;
- `grapher_mix_hash.py` e `grapher_mix_op.py`, utilizzati per confrontare i risultati tra diverse tipologie di dati.

Per ciascuna tipologia di dati sono stati generati:

- otto grafici relativi alle configurazioni della hashtable;
- nove grafici dedicati all'analisi delle singole operazioni.

In aggiunta, sono stati realizzati cinque grafici complessivi che mettono a confronto le prestazioni tra le diverse tipologie di dati analizzate.

Gli otto grafici relativi alla configurazione della hashtable sono:

- `latency_by_hash_strategy.pdf`
- `latency_by_max_load_factor.pdf`
- `latency_by_overflow_strategy.pdf`

- latency_by_resize_strategy.pdf
- memory_by_hash_strategy.pdf
- memory_by_max_load_factor.pdf
- memory_by_overflow_strategy.pdf
- memory_by_resize_strategy.pdf

I nove grafici relativi alle operazioni sono:

- key_distribution_by_overflow.pdf
- latency_by_operations_hash_strategy.pdf
- latency_by_operations_&_max_load_factor.pdf
- latency_by_operations_&_overflow_strategy.pdf
- latency_by_operations_&_resize_strategy.pdf
- latency_insert_exist_by_load_factor.pdf
- memory_by_operations_&_max_load_factor.pdf
- memory_by_operations_&_overflow_strategy.pdf
- memory_by_operations_&_resize_strategy.pdf

I cinque grafici relativi alle tipologie sono:

- latency_by_keytype_hash.pdf
- latency_by_operations_&_max_load_factor.pdf
- latency_by_keytype__overflow.pdf
- latency_by_keytype_resize.pdf
- memory_by_keytype.pdf
- latency_by_operation_keytype_elements.pdf

Listing 6.1: Esempio di script per la generazione di grafici

```

1  # -----
2  # Latenza / Overflow
3  # -----
4  df = pd.read_csv(path)
5
6  # -----
7  # Calcola media, minimo e massimo per gruppo
8  # -----
9  stats = (
10     df.groupby(["OVERFLOW", "Elements"])["Latenza"]
11     .agg(["mean", "min", "max"])
12     .reset_index()
13 )
14
15 # -----
16 # Crea la figura
17 # -----
18 plt.figure(figsize=(10,6))
19 colors = plt.cm.tab10.colors
20
21 # -----
22 # Plot per ogni gruppo di 'Elements'
23 # -----
24 x_labels = sorted(stats["OVERFLOW"].unique())
25 n_groups = len(stats["Elements"].unique())
26 width = 0.15
27
28 for i, elements in enumerate(sorted(stats["Elements"].
29     unique())):
30     subset = stats[stats["Elements"] == elements]
31
32     x_num = [x_labels.index(x_val) for x_val in subset["
33         OVERFLOW"]]
34     offset = (i - (n_groups-1)/2) * width
35     x_dodge = [x + offset for x in x_num]
36
37     y = subset["mean"]
38     err_low = y - subset["min"]
39     err_high = subset["max"] - y
40
41     plt.errorbar(
42         x_dodge,

```

```

41         y,
42         yerr=[err_low, err_high],
43         fmt="o",
44         label=f"Elements_{elements}",
45         color=colors[i % len(colors)],
46         capsize=4
47     )
48
49 # -----
50 # Migliora la leggibilità
51 # -----
52 plt.xticks(range(len(x_labels)), x_labels, rotation=30,
53            ha="right")
54 plt.xlabel("Overflow_Strategy")
55 plt.ylabel("Average_Latency[s]")
56 plt.title("Latency_by_Overflow_Strategy")
57 plt.grid(True, axis="y")
58 plt.legend()

```

6.2 Analisi dei dati

L'analisi dei dati si concentra principalmente su due metriche fondamentali:

- la latenza delle operazioni e dei task eseguiti;
- l'impatto sull'utilizzo della memoria.

L'obiettivo è valutare come le diverse configurazioni della hashtable influenzino le prestazioni complessive.

6.2.1 Hash Data

In questa sezione vengono analizzati i risultati relativi alle diverse configurazioni della hashtable. In particolare, si osserva l'influenza dei seguenti parametri:

- strategia di HASHING;
- fattore di carico massimo (*MAX LOAD FACTOR*);
- strategia di gestione delle collisioni (*OVERFLOW*);
- politica di ridimensionamento (*RESIZE*).

Latenza

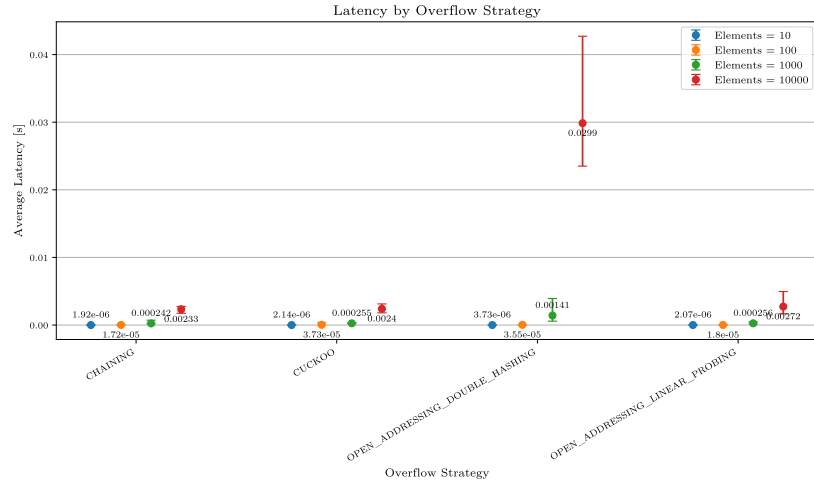


Figura 6.1: Latenza in funzione della strategia di gestione delle collisioni (Integer e potenza di 10)

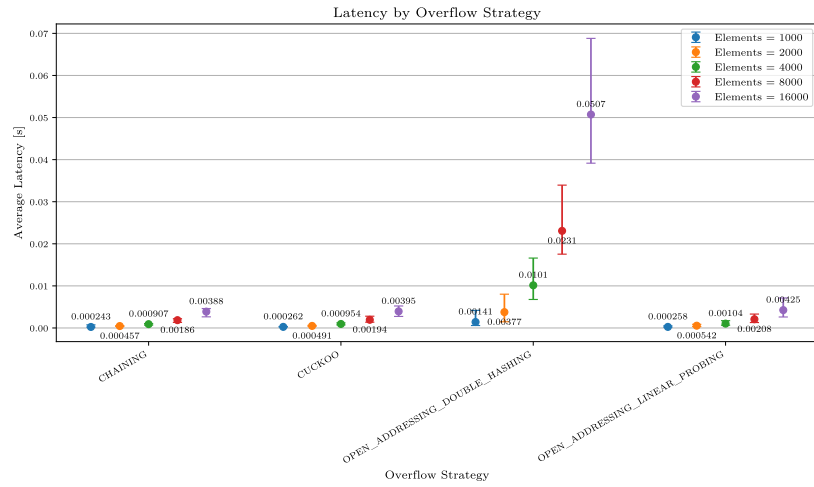


Figura 6.2: Latenza in funzione della strategia di gestione delle collisioni (Integer e doubling)

Dai grafici della latenza relativi alle strategie di OVERFLOW si evince che il tempo necessario a completare le operazioni rimane complessivamente simile al variare delle diverse tecniche. Un'eccezione è rappresentata dal metodo DOUBLE HASHING, che all'aumentare del numero di chiavi tende a peggiorare sia in termini di latenza media sia nei picchi massimi.

Tra i quattro metodi analizzati, CUCKOO e CHAINING mostrano inoltre una maggiore stabilità, con oscillazioni della latenza più contenute rispetto ai metodi di OPEN ADDRESSING.

Dal grafico generato dal doubling test inoltre si evince che la latenza circa raddoppia al raddoppiare della dimensione dell'input.

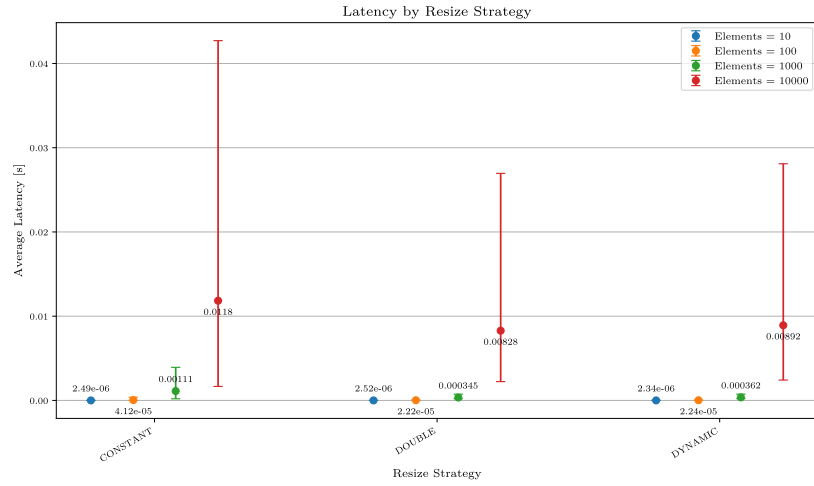


Figura 6.3: Latenza in funzione della strategia di RESIZE (Integer e potenza di 10)

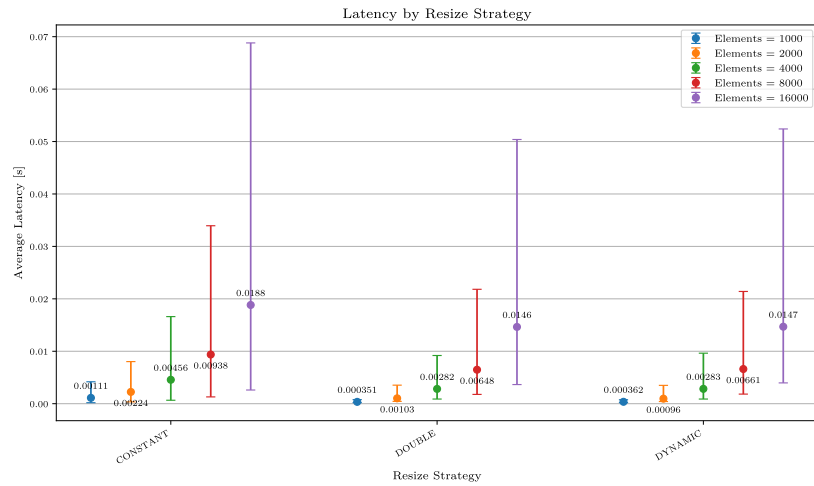


Figura 6.4: Latenza in funzione della strategia di RESIZE (Integer e doubling)

Per quanto concerne la latenza in relazione alle strategie di RESIZE, si osserva un comportamento coerente con le aspettative teoriche. La strategia CONSTANT risulta la meno efficiente: all'aumentare del numero di elementi, e quindi allontanandosi dalla dimensione iniziale fissata, le prestazioni peggiorano sensibilmente.

Le strategie DOUBLE e DYNAMIC presentano invece prestazioni complessivamente simili, con una latenza media leggermente migliore nel caso della strategia DOUBLE.

Dal grafico generato dal doubling test inoltre si evince che la latenza circa raddoppia al raddoppiare della dimensione dell'input.

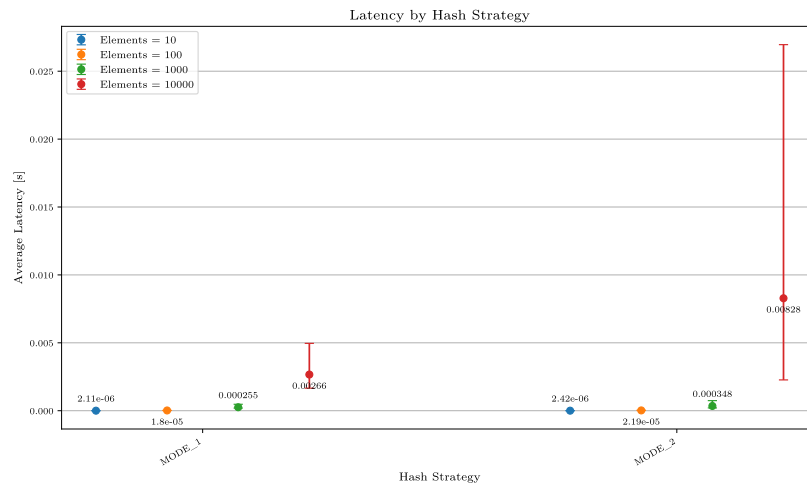


Figura 6.5: Latenza in funzione della strategia di HASHING (Integer e potenza di 10)

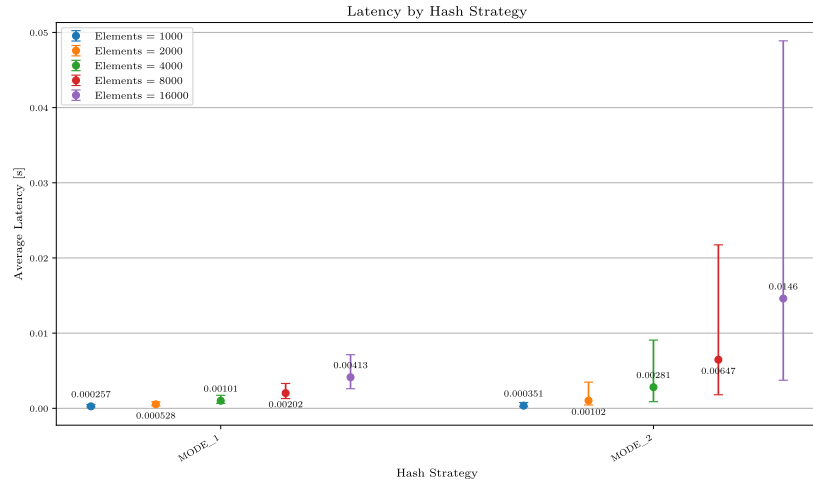


Figura 6.6: Latenza in funzione della strategia di HASHING (Integer e doubling)

Per quanto riguarda le due strategie di HASHING considerate, si osserva una differenza marcata: la prima risulta nettamente più efficiente in termini di latenza rispetto alla seconda.

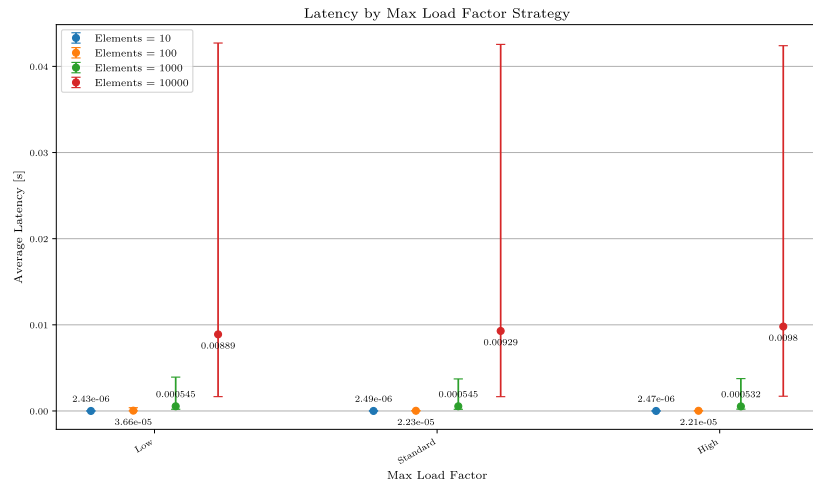


Figura 6.7: Latenza in funzione del MAX LOAD FACTOR (Integer e potenza di 10)

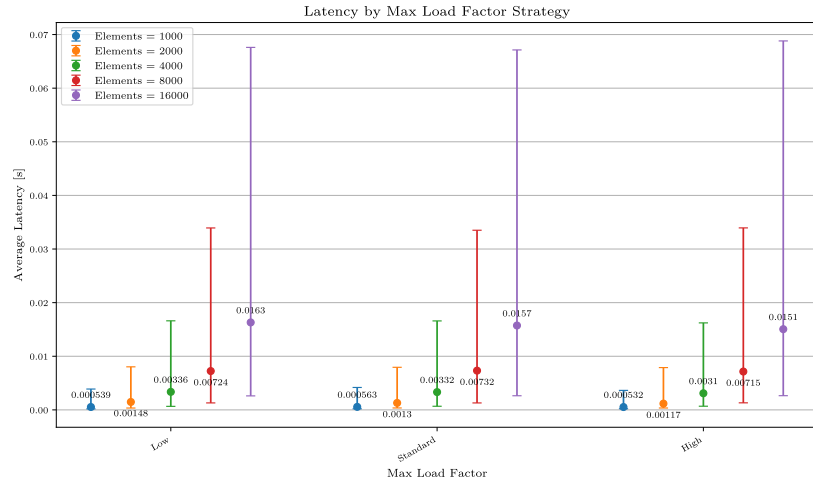


Figura 6.8: Latenza in funzione del MAX LOAD FACTOR (Integer e doubling)

Il MAX LOAD FACTOR sembra influire in maniera limitata sulla latenza complessiva: le differenze osservate tra le varie configurazioni risultano contenute.

Memoria

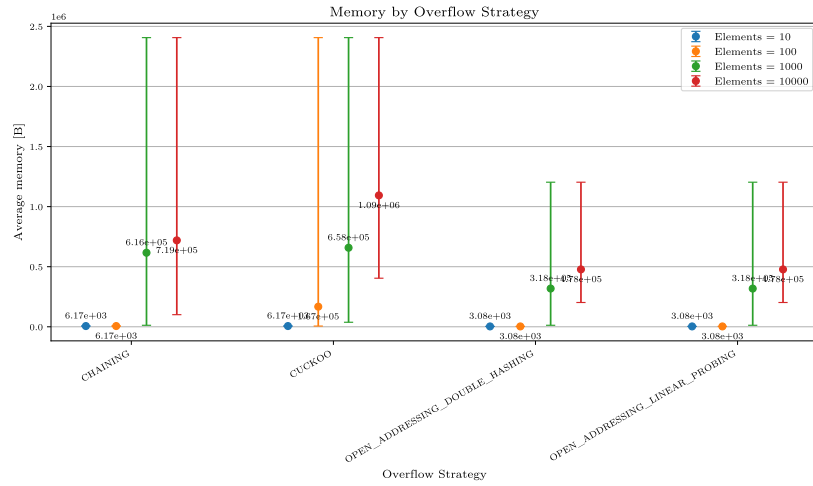


Figura 6.9: Utilizzo della memoria in funzione della strategia di gestione delle collisioni (Integer e potenza di 10)

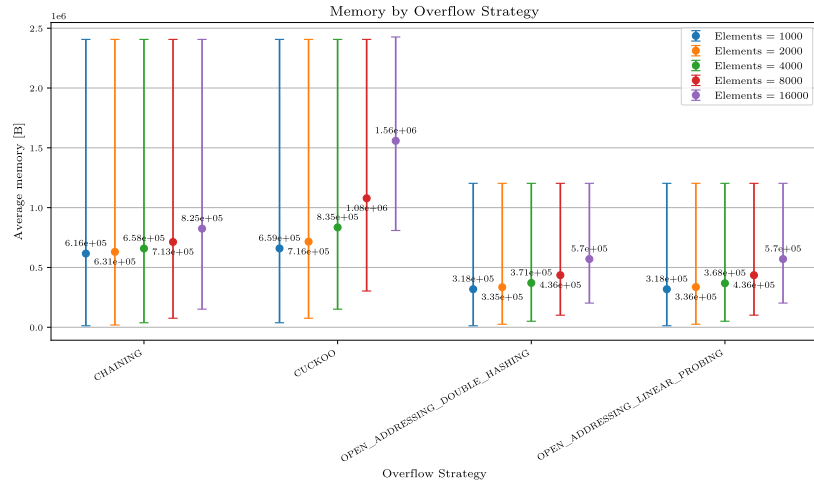


Figura 6.10: Utilizzo della memoria in funzione della strategia di gestione delle collisioni (Integer e doubling)

A differenza della latenza, l'utilizzo della memoria varia in modo significativo tra le diverse strategie. I metodi CHAINING e CUCKOO richiedono una quantità di memoria sensibilmente maggiore rispetto alle tecniche di OPEN ADDRESSING, sia in media sia nei picchi massimi delle singole esecuzioni.

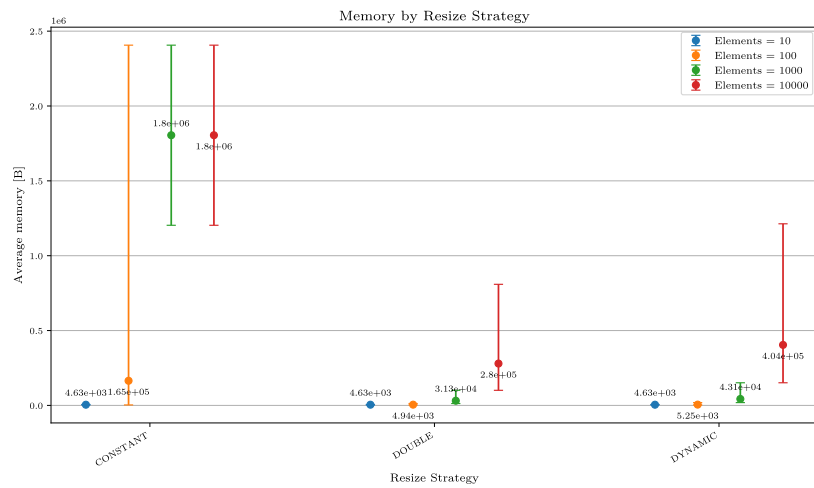


Figura 6.11: Utilizzo della memoria in funzione della strategia di RESIZE (Integer e potenza di 10)

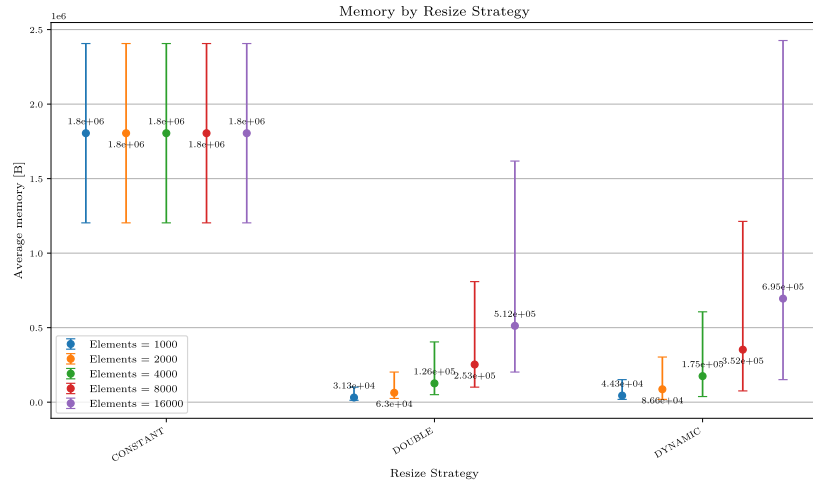


Figura 6.12: Utilizzo della memoria in funzione della strategia di RESIZE (Integer e doubling)

Per quanto riguarda le strategie di RESIZE, si osserva che il metodo CONSTANT tende a occupare più memoria rispetto agli altri due approcci. Tra DOUBLE e DYNAMIC, il primo mostra in media un comportamento leggermente più efficiente.

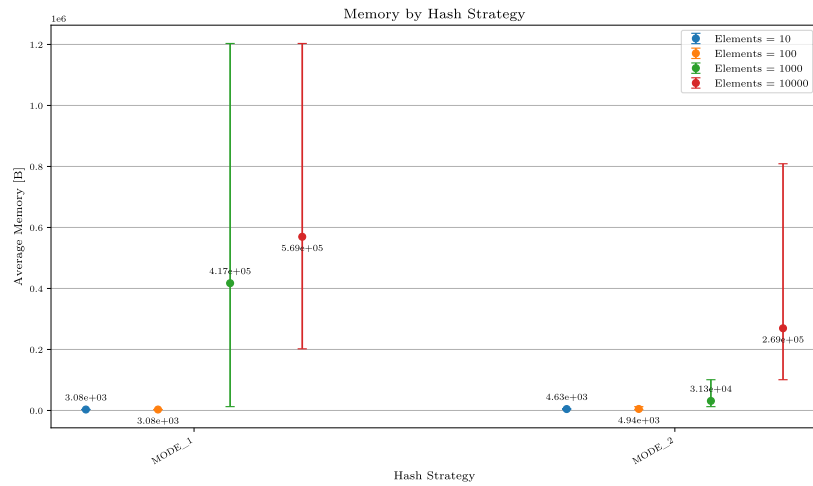


Figura 6.13: Utilizzo della memoria in funzione della strategia di HASHING (Integer e potenza di 10)

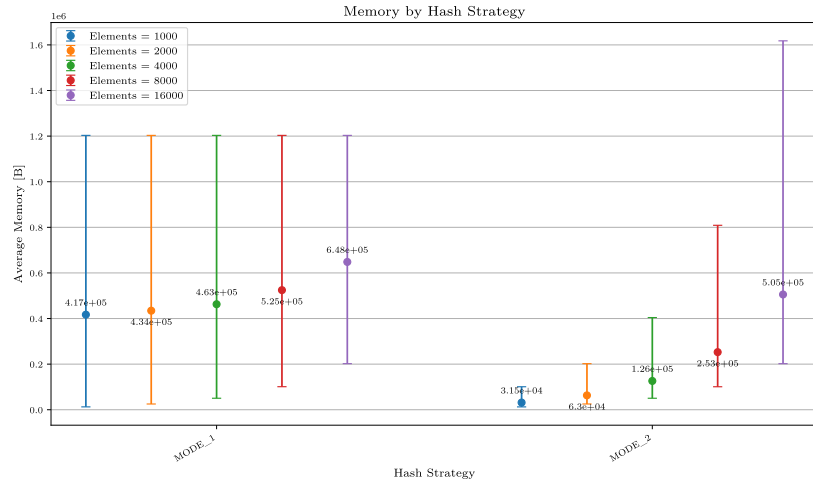


Figura 6.14: Utilizzo della memoria in funzione della strategia di HASHING (Integer e doubling)

La strategia di HASHING influisce in modo rilevante sull'utilizzo della memoria. In particolare, si osserva un comportamento opposto rispetto alla latenza: la seconda strategia risulta più efficiente della prima dal punto di vista dell'occupazione di memoria.

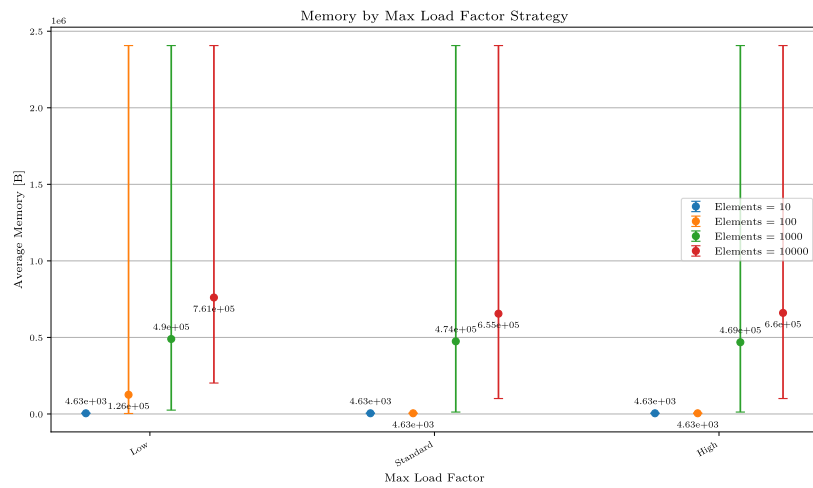


Figura 6.15: Utilizzo della memoria in funzione del MAX LOAD FACTOR (Integer e potenza di 10)

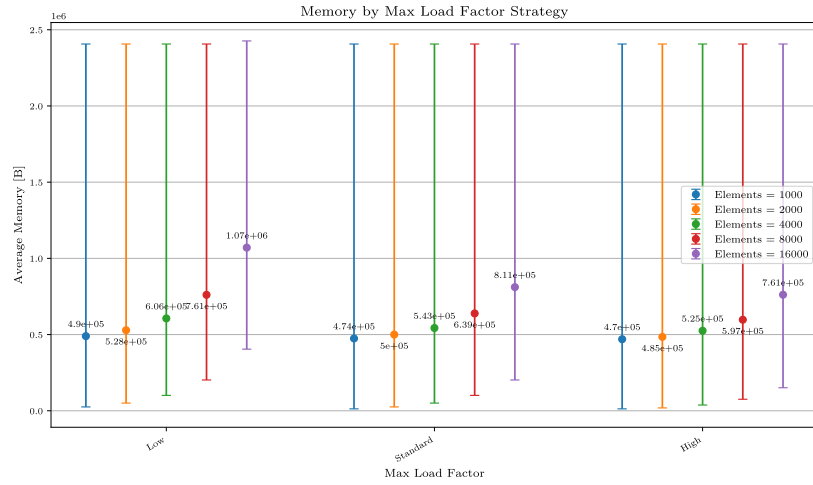


Figura 6.16: Utilizzo della memoria in funzione del MAX LOAD FACTOR (Integer e doubling)

Il MAX LOAD FACTOR, come atteso, incide in modo diretto sull'utilizzo della memoria: valori più bassi comportano un maggiore consumo di spazio, mentre configurazioni standard o elevate risultano più efficienti.

Sintesi dei risultati Dall'analisi complessiva emerge come le diverse configurazioni della hashtable influenzino in maniera differente le metriche considerate, evidenziando un compromesso tra latenza e utilizzo della memoria.

Per quanto riguarda la latenza, le strategie di OVERFLOW mostrano comportamenti generalmente simili, ad eccezione del DOUBLE HASHING, che tende a degradare all'aumentare del numero di chiavi. Le tecniche CUCKOO e CHAINING garantiscono invece una maggiore stabilità. Tra le politiche di RESIZE, la strategia CONSTANT risulta inefficiente, mentre DOUBLE e DYNAMIC offrono prestazioni comparabili, con un leggero vantaggio per DOUBLE. La scelta della funzione di HASHING risulta determinante, mentre il MAX LOAD FACTOR ha un impatto limitato. Generalmente inoltre si evince un andamento della latenza che tende a raddoppiare al raddoppio della dimensione dell'input.

Dal punto di vista della memoria, emergono differenze più marcate: CHAINING e CUCKOO risultano più costosi rispetto all'OPEN ADDRESSING, mentre la strategia CONSTANT comporta un maggiore utilizzo di memoria rispetto alle alternative. Inoltre, la seconda strategia di HASHING si dimostra più efficiente, evidenziando un comportamento opposto rispetto alla latenza.

In sintesi, non esiste una configurazione universalmente ottimale: la scelta dipende dal contesto applicativo e dai vincoli specifici. Tuttavia, l'utilizzo della strategia DOUBLE per il RESIZE e una scelta accurata della funzione di HASHING rappresentano, in generale, un buon compromesso tra efficienza e stabilità.

6.2.2 Operation Data

Questa sezione è dedicata all'analisi delle singole operazioni sulla hashtable (inserimento, ricerca, aggiornamento e cancellazione). Viene studiato il comportamento delle operazioni al variare delle diverse configurazioni, con particolare attenzione a:

- distribuzione delle chiavi;
- strategia di hashing;
- fattore di carico massimo (*max load factor*);
- strategia di gestione delle collisioni (*overflow*);
- politica di ridimensionamento (*resize*).

Latenza

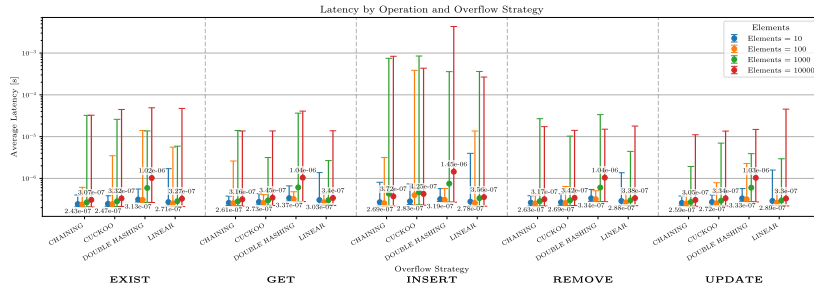


Figura 6.17: Latenza per operazione in funzione della strategia di gestione delle collisioni (Integer e potenza di 10)

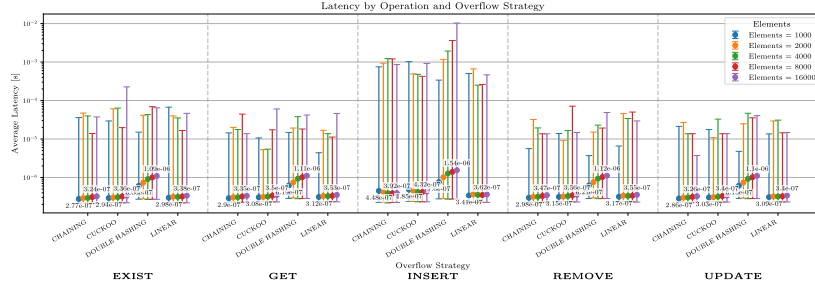


Figura 6.18: Latenza per operazione in funzione della strategia di gestione delle collisioni (Integer e doubling)

Dal grafico si osserva come l'operazione di *insert* presenti generalmente una latenza superiore rispetto alle altre. Ciò è dovuto al controllo del *load factor* e alla possibile necessità di effettuare operazioni di *resize*, che introducono un costo aggiuntivo.

Tra le diverse strategie di gestione delle collisioni, il *double hashing* mostra prestazioni significativamente peggiori su tutte le operazioni, in particolare sull'inserimento, dove il costo delle sonde multiple incide maggiormente.

Le strategie *cuckoo hashing* e *chaining* evidenziano prestazioni complessivamente simili. Tuttavia, si nota che:

- il *cuckoo hashing* presenta picchi leggermente più contenuti per l'operazione di *insert* all'aumentare del numero di chiavi;
- il *chaining* risulta leggermente più stabile nelle operazioni di ricerca, aggiornamento e cancellazione.

L'*open addressing* con probing lineare, pur mostrando una latenza media comparabile alle precedenti strategie, presenta prestazioni peggiori per dataset di piccole dimensioni. Tuttavia, al crescere del numero di chiavi, mostra una maggiore stabilità e una crescita più controllata della latenza.

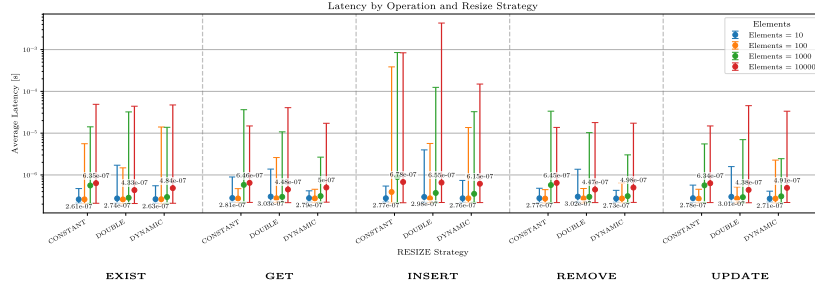


Figura 6.19: Latenza per operazione in funzione della strategia di ridimensionamento (Integer e potenza di 10)

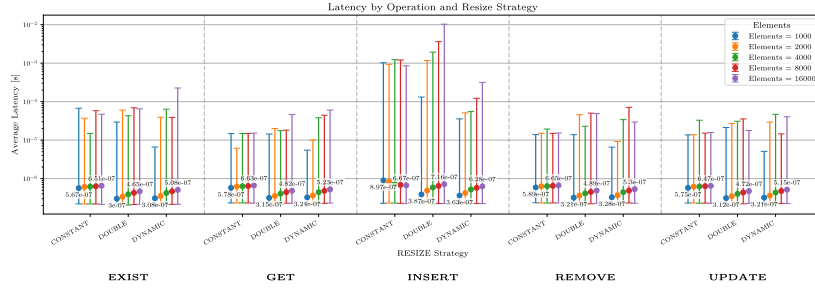


Figura 6.20: Latenza per operazione in funzione della strategia di ridimensionamento (Integer e doubling)

L'analisi delle politiche di *resize* evidenzia come la strategia *constant* comporti latenze medie e picchi significativamente superiori rispetto alle altre. Questo è dovuto alla maggiore frequenza di ridimensionamenti o alla loro scarsa efficienza.

Le strategie *double* e *dynamic* mostrano invece prestazioni medie simili. Tuttavia:

- la strategia *dynamic* garantisce una maggiore stabilità complessiva;
- la strategia *double* presenta picchi più marcati, soprattutto all'aumentare del numero di chiavi.

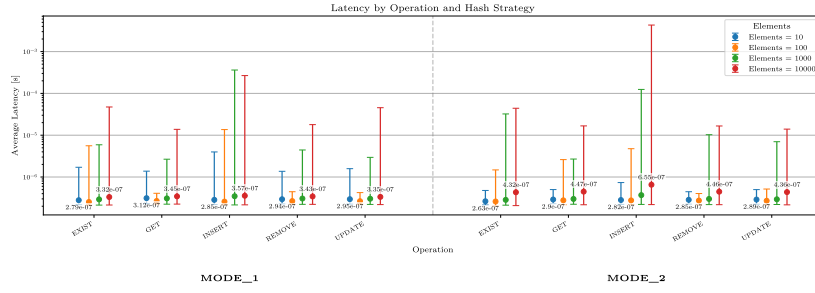


Figura 6.21: Latenza per operazione in funzione della funzione di hash (Integer e potenza di 10)

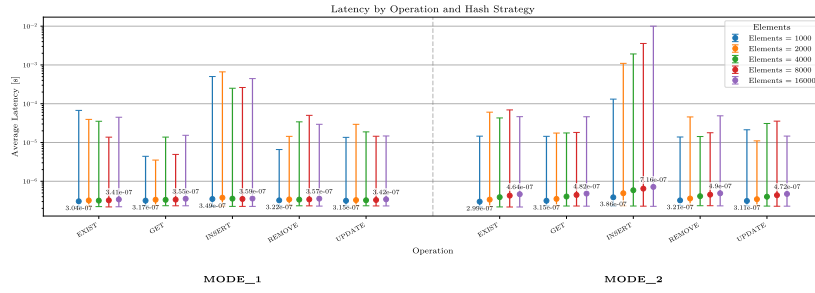


Figura 6.22: Latenza per operazione in funzione della funzione di hash (Integer e doubling)

Il confronto tra le due funzioni di hash mostra che la prima strategia risulta più efficiente sia in termini di latenza media sia nei picchi massimi, soprattutto su dataset di grandi dimensioni.

La seconda funzione di hash, pur essendo meno performante in media, mostra risultati leggermente migliori nel caso di dataset molto piccoli, dove il costo computazionale della funzione ha un impatto più rilevante rispetto alla distribuzione delle chiavi.

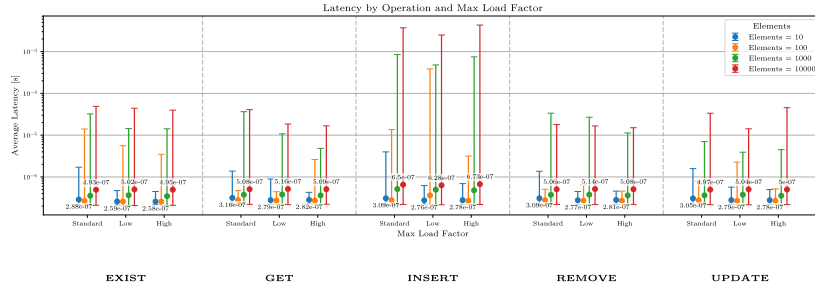


Figura 6.23: Latenza per operazione in funzione del max load factor (Integer e potenza di 10)

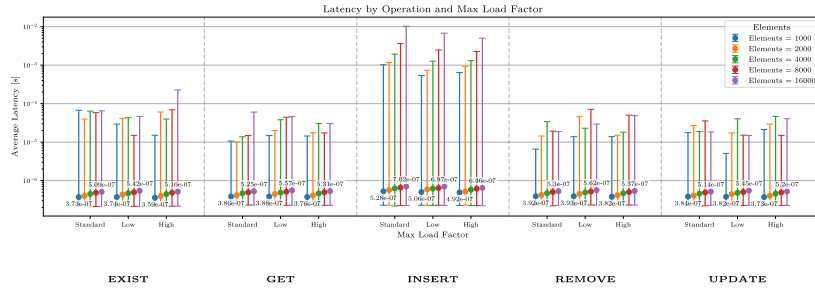


Figura 6.24: Latenza per operazione in funzione del max load factor (Integer e doubling)

Le prestazioni in funzione del *max load factor* risultano nel complesso simili tra le diverse configurazioni. Tuttavia, emergono alcune differenze:

- un *load factor* basso (*LOW*) garantisce maggiore stabilità nelle varie operazioni;
- valori più elevati (*STANDARD* e *HIGH*) offrono migliori prestazioni medie nell'operazione di insert, grazie a un minor numero di ridimensionamenti.

Questo evidenzia il classico trade-off tra utilizzo della memoria e prestazioni.

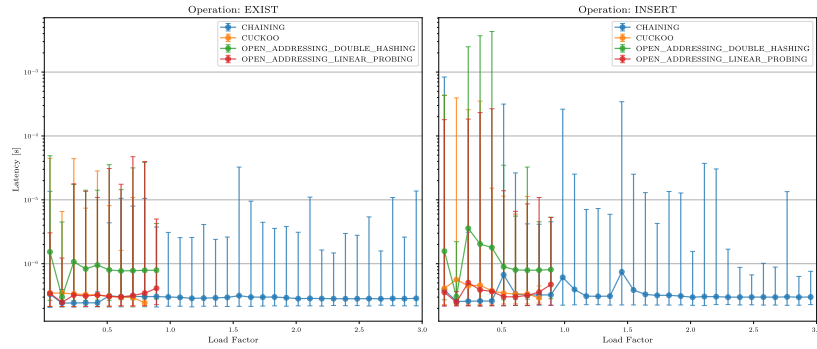


Figura 6.25: Latenza delle operazioni di insert e get al variare del load factor e della strategia di overflow (Integer e potenza di 10)

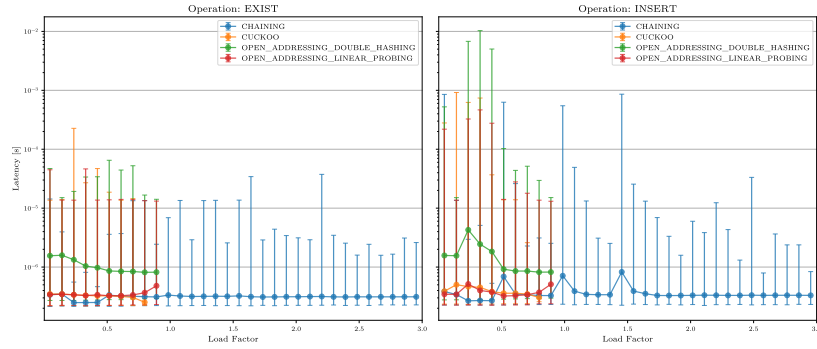


Figura 6.26: Latenza delle operazioni di insert e get al variare del load factor e della strategia di overflow (Integer e doubling)

Le strategie *chaining* e *cuckoo hashing* mantengono latenze inferiori e più stabili, mentre il *double hashing* continua a mostrare prestazioni peggiori.

In generale, l'aumento del *load factor* tende a penalizzare maggiormente le tecniche basate su *open addressing*, a causa dell'incremento delle collisioni e del numero medio di sonde necessarie.

Sintesi dei risultati L'analisi della latenza delle operazioni evidenzia come le prestazioni di una hashtable dipendano in modo significativo dall'interazione tra strategia di gestione delle collisioni, politica di ridimensionamento e valore del *max load factor*.

In generale, le operazioni di inserimento risultano le più costose, principalmente a causa del controllo del fattore di carico e delle eventuali operazioni di *resize*. Tra le strategie di *overflow*, il *double hashing* si dimostra la meno

efficiente, mostrando latenze elevate e scarsa scalabilità. Al contrario, *chaining* e *cuckoo hashing* offrono un buon compromesso tra prestazioni medie e stabilità, mentre il probing lineare risulta più sensibile alla dimensione del dataset ma garantisce una crescita della latenza più regolare.

Per quanto riguarda il *resize*, strategie adattive come la *dynamic* permettono di contenere i picchi di latenza, risultando complessivamente più robuste rispetto a strategie più semplici come la *constant*. Anche la scelta della funzione di hash incide in modo rilevante: una buona distribuzione delle chiavi diventa sempre più critica all'aumentare della dimensione dei dati.

Infine, il *max load factor* introduce un chiaro compromesso tra efficienza temporale e utilizzo della memoria: valori bassi migliorano la stabilità e riducono le collisioni, mentre valori più elevati riducono la frequenza dei ridimensionamenti ma possono degradare le prestazioni nelle tecniche di *open addressing*.

Complessivamente, i risultati suggeriscono che non esiste una configurazione ottimale in senso assoluto, ma che la scelta dei parametri debba essere guidata dal contesto applicativo, dal volume dei dati e dai requisiti di prestazione.

6.2.3 Mixed Data

In questa sezione vengono analizzati cinque grafici complessivi che confrontano direttamente le prestazioni delle diverse tipologie di dati.

L'obiettivo è valutare il comportamento della hashtable al variare del tipo di input, evidenziando eventuali differenze in termini di latenza e utilizzo della memoria.

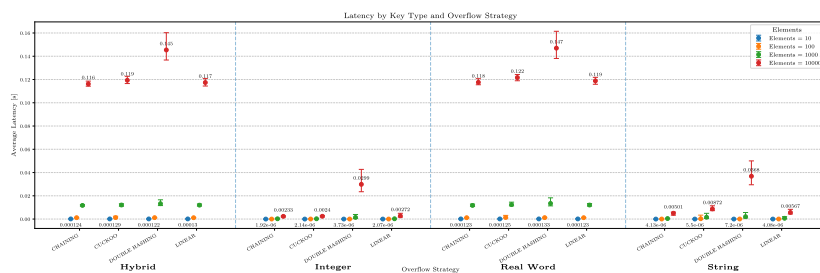


Figura 6.27: Latenza per operazione in funzione della strategia di gestione delle collisioni, suddivisa per tipo di chiave. (Integer e potenza di 10)

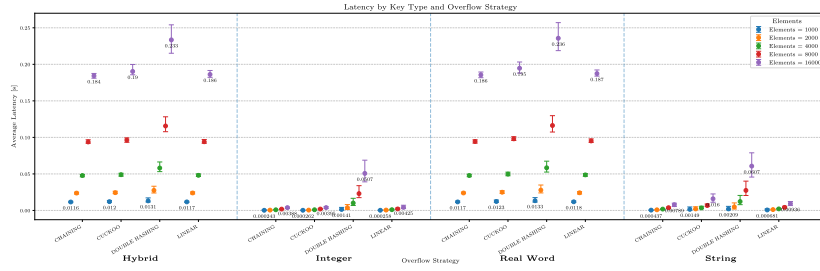


Figura 6.28: Latenza per operazione in funzione della strategia di gestione delle collisioni, suddivisa per tipo di chiave. (Integer e doubling)

Dal grafico emerge chiaramente che le chiavi di tipo *Integer* garantiscono le migliori prestazioni complessive. Le *String* generate casualmente presentano latenze leggermente superiori, ma la differenza rispetto agli *Integer* è minima. Al contrario, le chiavi basate su *Real Words* mostrano latenze significativamente più elevate, probabilmente a causa di una distribuzione meno uniforme che genera più collisioni.

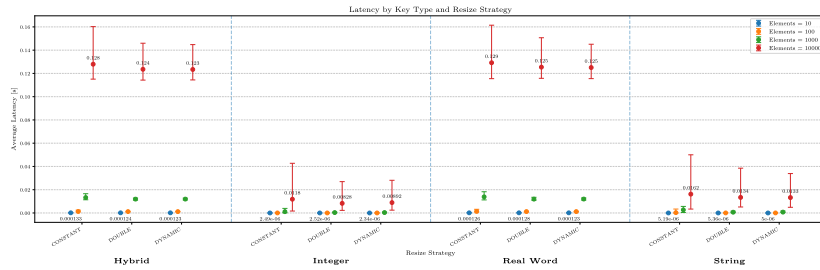


Figura 6.29: Latenza per operazione in funzione della strategia di *resize*, suddivisa per tipo di chiave. (Integer e potenza di 10)

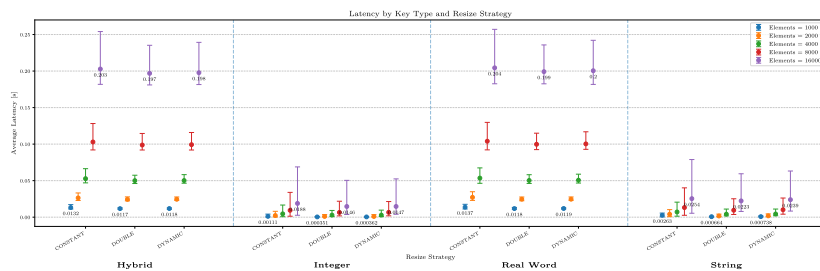


Figura 6.30: Latenza per operazione in funzione della strategia di *resize*, suddivisa per tipo di chiave. (Integer e doubling)

Risultati analoghi si osservano analizzando la strategia di *resize*. Gli *Integer* mantengono prestazioni superiori, seguiti dalle *String* casuali. Le chiavi basate su *Real Words* mostrano latenze sensibilmente più alte, confermando il maggiore impatto delle collisioni su dataset meno uniformi.

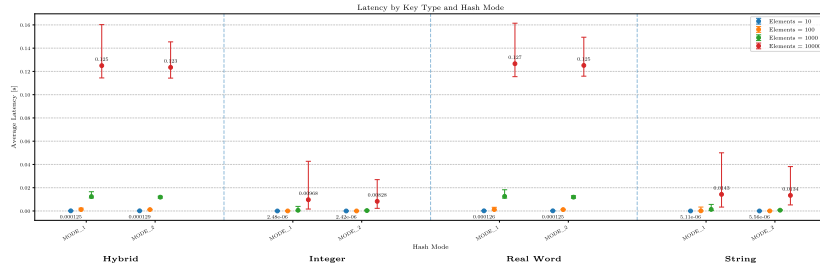


Figura 6.31: Latenza per operazione in funzione della funzione di hash, suddivisa per tipo di chiave. (Integer e potenza di 10)

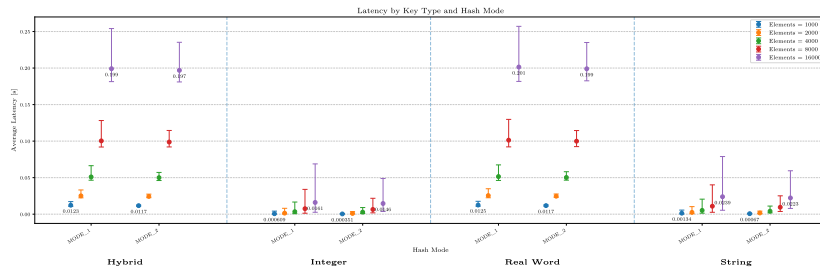


Figura 6.32: Latenza per operazione in funzione della funzione di hash, suddivisa per tipo di chiave. (Integer e doubling)

Anche la scelta della funzione di hash evidenzia la stessa tendenza: le chiavi *Integer* ottimizzano la distribuzione, riducendo le collisioni e quindi la latenza, mentre le *Real Words* risultano più penalizzate.

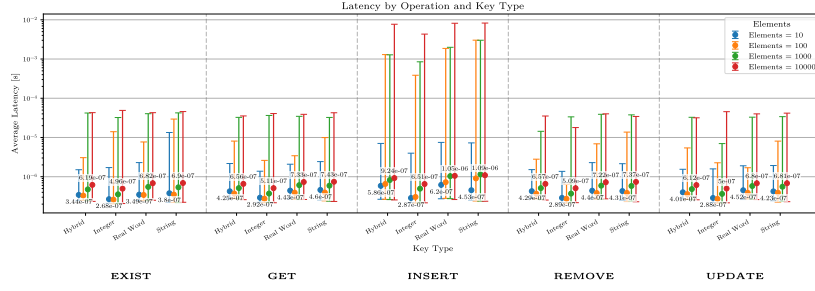


Figura 6.33: Latenza per singola operazione in funzione del tipo di chiave e del numero di elementi. (Integer e potenza di 10)

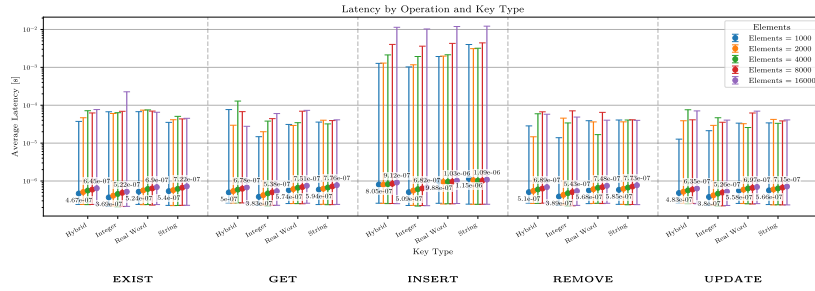


Figura 6.34: Latenza per singola operazione in funzione del tipo di chiave e del numero di elementi. (Integer e doubling)

A livello delle singole operazioni, le prestazioni confermano quanto osservato precedentemente. Si nota un leggero vantaggio dell'approccio ibrido (*Hybrid*) nell'operazione di *insert*, rispetto alle strategie full stringa, grazie a una combinazione più efficiente di tipi di chiave.

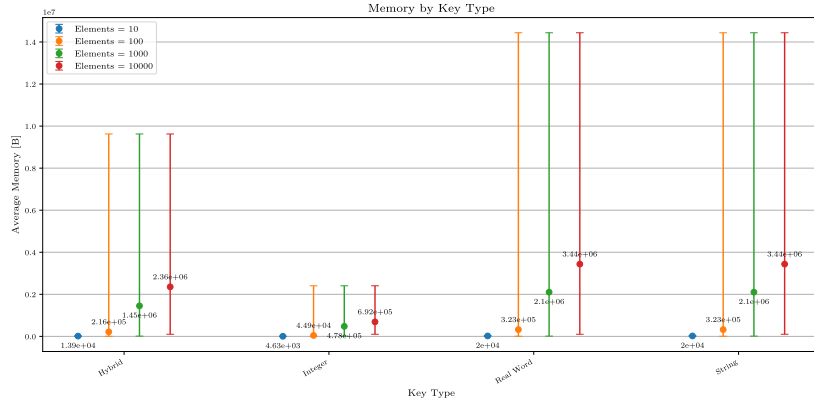


Figura 6.35: Memoria media utilizzata in funzione del tipo di chiave. (Integer e potenza di 10)

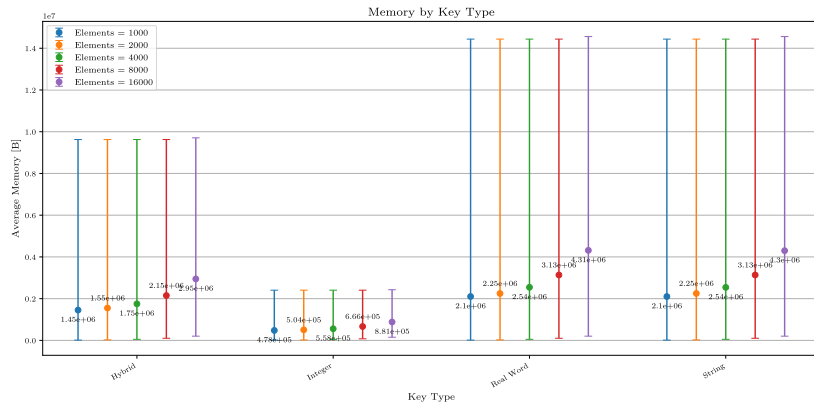


Figura 6.36: Memoria media utilizzata in funzione del tipo di chiave. (Integer e doubling)

L'analisi della memoria evidenzia che gli *Integer* richiedono meno spazio, seguiti dall'approccio ibrido, mentre le *String* complete risultano le più costose in termini di consumo di memoria.

Sintesi dei risultati Dall'analisi dei grafici emerge chiaramente come il tipo di chiave influisca in modo determinante sulle prestazioni della hashtable. Gli *Integer* offrono i migliori risultati sia in termini di latenza sia di memoria utilizzata, grazie alla loro natura semplice e alla distribuzione uniforme. Le *String* casuali mantengono prestazioni simili ma leggermente peggiori, mentre le chiavi basate su *Real Words* risultano più penalizzate a causa di collisioni più frequenti e maggiore overhead di memoria.

L'approccio ibrido rappresenta un buon compromesso, combinando la compattezza degli *Integer* con la flessibilità delle *String*, offrendo un leggero miglioramento nelle operazioni di inserimento rispetto alle strategie full stringa.

In generale, i risultati suggeriscono che la scelta del tipo di chiave deve essere guidata dal contesto applicativo e dai requisiti di prestazione, privilegiando tipologie più compatte e uniformi quando si cerca massima efficienza.

Capitolo 7

Conclusioni

Il lavoro svolto ha permesso di analizzare in maniera sistematica le prestazioni delle hashtable in relazione a diversi tipi di chiave, configurazioni e politiche di gestione delle collisioni e del ridimensionamento. L'approccio adottato ha combinato raccolta dati sperimentali, rielaborazione tramite Python e visualizzazione grafica, permettendo di confrontare in modo chiaro e dettagliato i risultati ottenuti.

Dall'analisi emerge come le prestazioni di una hashtable dipendano in maniera significativa da molteplici fattori, tra cui:

- il tipo di chiave utilizzata (*Integer*, *String* casuali, *Real Words*, o approccio *Hybrid*);
- la strategia di gestione delle collisioni (*open addressing*, *chaining*, *cuckoo hashing*, ecc.);
- la politica di ridimensionamento (*resize*) e il fattore di carico massimo (*max load factor*);
- la funzione di hash scelta per distribuire le chiavi.

In particolare, le osservazioni principali possono essere sintetizzate come segue:

- **Tipo di chiave:** le chiavi di tipo *Integer* garantiscono le migliori prestazioni sia in termini di latenza sia di utilizzo della memoria, grazie alla loro semplicità e alla distribuzione uniforme. Le *String* casuali presentano prestazioni leggermente inferiori, mentre le *Real Words* risultano più penalizzate. L'approccio *Hybrid* si conferma un buon compromesso, combinando efficienza e flessibilità.

- **Strategia di gestione delle collisioni:** le tecniche di *chaining* e *cuckoo hashing* offrono prestazioni stabili e affidabili, mentre il *double hashing* tende a mostrare latenze maggiori, soprattutto con dataset di grandi dimensioni. L'*open addressing* è più sensibile al numero di chiavi, ma garantisce una crescita controllata della latenza.
- **Politica di ridimensionamento e max load factor:** strategie adattive come *dynamic resize* permettono di contenere i picchi di latenza, risultando più robuste rispetto a strategie fisse come *constant*. Valori elevati del *max load factor* migliorano le prestazioni medie riducendo i ridimensionamenti, mentre valori bassi aumentano la stabilità a discapito della memoria.
- **Funzione di hash:** la scelta della funzione di hash influenza significativamente la distribuzione delle chiavi e, di conseguenza, la latenza delle operazioni. Funzioni più efficienti garantiscono una minor incidenza delle collisioni e una maggiore uniformità nelle prestazioni.

In sintesi, non esiste una configurazione universalmente ottimale: la scelta della struttura e dei parametri della hashtable deve essere guidata dal contesto applicativo, dal volume dei dati e dai requisiti di prestazione. I risultati ottenuti evidenziano come combinare chiavi compatte e uniformi, una politica di ridimensionamento efficiente e una funzione di hash appropriata rappresenti il compromesso ideale tra latenza e utilizzo della memoria.

Capitolo 8

Uso delle LLM

Nel presente progetto, l'impiego degli LLM è stato circoscritto a un ruolo di supporto, principalmente orientato alla consultazione rapida di documentazione tecnica e manuali. In particolare, tali strumenti sono stati utilizzati per reperire informazioni utili alla progettazione e implementazione in C++ della classe template e della realizzazione dei grafici in python, contribuendo a velocizzare il processo di sviluppo e a chiarire eventuali dubbi sintattici o strutturali legati al linguaggio.

Inoltre, i modelli di linguaggio sono stati impiegati per la revisione del testo della relazione, fornendo suggerimenti per la correzione grammaticale, sintattica e stilistica. Questo ha consentito di ottenere un elaborato più chiaro, coerente e formalmente corretto, mantenendo comunque il pieno controllo sui contenuti.

Bibliografia

- [1] w3schools. *C++ Templates*. 2026. URL: https://www.w3schools.com/cpp/cpp_templates.asp.
- [2] Wikipedia. *Cuckoo hashing*. 2026. URL: https://en.wikipedia.org/wiki/Cuckoo_hashing.
- [3] Wikipedia. *Hash table*. 2026. URL: https://en.wikipedia.org/wiki/Hash_table.